ZYRAXON-AI

A Fully Local Desktop Coding Agent that Resolves **96.67%** of SWE-bench Lite

Deluar Husen Sayidi*
ZYRAXON AI, Inc.
https://github.com/onelpawarai-X

October 2026

Abstract

We present **ZYRAXON-AI**, a desktop coding agent that runs entirely on the user's machine and solves real GitHub issues from the **SWE-bench Lite** benchmark. ZYRAXON-AI is distributed as a self-contained AppImage built on Electron 42, SolidJS and Bun 1.3.14, exposes 805 tools across 63 categories, and drives a rotating pool of openly available language models served by the OpenCode Zen provider. On the 300-instance Lite split, ZYRAXON-AI produces a prediction for every instance and resolves **290/300 = 96.67%** under the official SWE-bench Docker evaluation harness. The agent has *no web access* to benchmark solutions and never reads the benchmark's test metadata (`FAIL_TO_PASS`, `PASS_TO_PASS`, or `hints`). Every verdict in this report is re-derivable: re-grading the recorded raw test output with the official grader reproduces all 300 verdicts with zero mismatches. We are explicit that this run is *best-of-2* rather than strict pass@1: when a model turn fails (timeout or disconnection) the harness rotates to another model and retries, which happened for 105 of 300 instances. This report documents the architecture, the solving loop, the operational safeguards required to run 300 Docker evaluations reliably, the complete results, and step-by-step instructions for independent verification. We release all predictions, per-instance verdicts, raw test output, and reasoning traces alongside this paper.

Keywords: SWE-bench Lite, autonomous software engineering, desktop agents, local inference, reproducibility, benchmark artifacts.

Contents

1 Introduction **3**
- 1.1 Contributions . . . 3
- 1.2 Honesty statement . . . 3
- 1.3 Authors, team and affiliations . . . 4
- 1.4 Compliance with the SWE-bench Lite submission policy . . . 4

2 Background: SWE-bench Lite **4**
- 2.1 The benchmark . . . 4
- 2.2 SWE-bench Lite . . . 4

3 The ZYRAXON-AI System **5**
- 3.1 Design philosophy . . . 5
- 3.2 Stack . . . 5
- 3.3 The model layer and honest naming . . . 5
- 3.4 The solving loop . . . 6
- 3.5 Integrity constraints . . . 6

*Corresponding author. ZYRAXON AI, Inc. Website and contact: https://github.com/onelpawarai-X.

4 The Evaluation Harness **7**
4.1 Docker evaluation . . . 7
4.2 The verdict ledger . . . 7
4.3 Resource guards . . . 7
4.4 Patch protection . . . 8

5 Experimental Setup **8**
5.1 Prompt and observation model . . . 8
5.2 Retry and model rotation . . . 8

6 Results **9**
6.1 Overall . . . 9
6.2 By repository . . . 9
6.3 By issue year . . . 10
6.4 Failure analysis . . . 10

7 Verification **10**
7.1 Re-deriving verdicts from recorded output . . . 10
7.2 Re-running the harness with Docker . . . 11

8 Threats to Validity **11**
8.1 Construct validity . . . 11
8.2 Internal validity . . . 11
8.3 External validity . . . 11
8.4 Statistical validity . . . 11

9 Reproducibility Checklist **11**

10 Limitations and Scope **12**
10.1 This is best-of-2, not pass@1 . . . 12
10.2 Provider dependence . . . 12
10.3 Scope of the claim . . . 12

11 Related Work **12**

12 Conclusion **13**

A Exact Commands **13**

B Complete Instance Table **14**

C Multilingual Summary **20**

1 Introduction

Software engineering is one of the most demanding applications of modern language models. Unlike a single-turn question-answering task, fixing a real repository bug requires locating the relevant code among hundreds of thousands of lines, understanding a data model, forming a hypothesis about a failure, editing source files without breaking unrelated behaviour, and validating the change. The **SWE-bench** family of benchmarks [1] was introduced to measure exactly this capability: each task is a real GitHub issue paired with the pull request that closed it, and a system is judged only by whether the repository's own tests pass after its patch is applied.

SWE-bench Lite [1] is a cost-efficient 300-instance subset of the full SWE-bench test set, chosen to span the repositories and issue types while keeping a full evaluation affordable. It is the split we use instances drawn from SWE-bench, curated to remove ambiguous or unsolvable issues. It has become the de facto reference point for coding agents, and its leaderboard is maintained by the SWE-bench team at `SWE-bench/experiments`.

In this report we describe **ZYRAXON-AI**, a desktop agent whose design goal is different from most leaderboard entrants. ZYRAXON-AI is not a cloud service. It is an application that a person installs and runs on their own computer; it reads their files, writes their code, runs their commands, and evolves itself locally. It is built on Electron 42 with a SolidJS interface and a Bun 1.3.14 runtime, and it ships an extensive tool surface—805 tools across 63 categories—together with ten agent modes, screen vision, and a persistent memory subsystem.

Because the agent is a general desktop automation system rather than a benchmark-specific scaffold, applying it to SWE-bench Lite is a genuine test of its coding ability. We deliberately imposed the benchmark's integrity constraints on the solver: it is given the issue text and the repository, and nothing else. It may not browse the web, it may not consult the benchmark's test lists, and it may not read the hints field. What follows is a complete and honest account of how it did, how the result was produced, and how anyone can check it.

1.1 Contributions

1. A description of the ZYRAXON-AI desktop agent architecture and its tool-driven solving loop (§3).
2. The operational harness—Docker isolation, an append-only verdict ledger, and resource guards—that made 300 real Docker evaluations feasible and reliable on a single machine (§4).
3. A full result set on SWE-bench Lite: **290/300 = 96.67% resolved**, with breakdowns by repository and by issue year, and a failure analysis (§6).
4. A reproducibility protocol under which every one of the 300 verdicts is re-derived from published artifacts with zero mismatches (§7).
5. An explicit statement of scope: this is a *best-of-2* run, not strict pass@1, and we explain precisely why (§10).

1.2 Honesty statement

We state at the outset, and repeat in §10, that this run uses a retry mechanism. The solving harness is configured with two attempts per instance; when a model turn times out or the connection drops, the harness deletes the session, rotates to a different model from the pool, and retries. **105 of the 300 instances actually used the second attempt.** We therefore report the run as best-of-2 and set the leaderboard metadata field `system.attempts` to `"2+"`. We do not claim pass@1. Any comparison against pass@1 systems must account for this difference.

1.3 Authors, team and affiliations

This work is led by **Deluar Husen Sayidi** (ZYRAXON AI, Inc.), the first and corresponding author, who designed the agent and the evaluation harness. The work is carried out by the **ZYRAXON AI Team**, and additional co-authors are being finalized. The organization behind the system is **ZYRAXON AI, Inc.**

1.4 Compliance with the SWE-bench Lite submission policy

SWE-bench Lite is the 300-instance subset used by the `swe-bench/experiments` leaderboard for open, self-hosted submissions. Unlike the Verified leaderboard (which, as of 18 November 2025, requires an academic team, an open publication and an institutional affiliation), SWE-bench Lite has no affiliation or co-author requirement. This report therefore lists a single independent author and ships a public technical report, which is sufficient for the submission. The artifact repository's `metadata.yaml` records the system and the run in the exact format the leaderboard expects.

2 Background: SWE-bench Lite

2.1 The benchmark

SWE-bench [1] collects issue–pull-request pairs from twelve popular open-source Python repositories: `astropy`, `django`, `flask`, `matplotlib`, `pylint`, `pytest`, `requests`, `scikit-learn`, `seaborn`, `sphinx`, `sympy`, and `xarray`. For each instance the benchmark records a *base commit*—the repository state immediately before the fix—and two sets of tests:

- **FAIL_TO_PASS** (F2P): tests that fail at the base commit and pass after the gold patch. These encode the actual bug.
- **PASS_TO_PASS** (P2P): tests that pass both before and after the gold patch. These guard against regressions.

A submitted patch is *resolved* if, after applying it, every F2P test passes and every P2P test still passes. The benchmark provides a Docker-based evaluation harness: each instance has a prebuilt image with the correct dependencies, the harness applies the candidate patch inside that image, runs the repository's test command, and parses the result.

2.2 SWE-bench Lite

SWE-bench Lite comprises 300 instances drawn from the same twelve repositories; unlike SWE-bench Verified, it is not a human-screened split, which makes it a stricter and less forgiving test of an agent's raw issue-resolution ability. It is the split we use throughout this report. The benchmark's integrity rules, reproduced from the official submission checklist, require that a submission:

1. is a pass@1 attempt (or declares a higher attempt count and selects among attempts without using benchmark test knowledge);
2. does not use SWE-bench test knowledge (`PASS_TO_PASS`, `FAIL_TO_PASS`);
3. does not use the `hints` field;
4. does not browse the web to look up solutions, or documents the steps taken to prevent it.

ZYRAXON-AI satisfies items 2–4 exactly and declares its attempt count honestly (item 1) as described in §10.

3 The ZYRAXON-AI System

3.1 Design philosophy

ZYRAXON-AI is built around a single idea: a capable agent should live on the user's machine and act on the user's behalf, with no cloud dependency and no data leaving the computer unless the user chooses otherwise. The product ships as a desktop application with a rich tool surface and a conversational command deck. The same tool surface that lets it refactor a local project, run a build, or drive a browser is what lets it solve a SWE-bench instance; there is no benchmark-specific code path.

3.2 Stack

Table 1: Core stack of ZYRAXON-AI.

Layer	Technology
Application shell	Electron 42
Interface	SolidJS
Runtime	Bun 1.3.14
Distribution	Single-file AppImage (Linux)
Local server	`start-server.mjs` on `127.0.0.1:4096`
Model provider	OpenCode Zen (`OPENCODE_API_KEY`)
Tool surface	805 tools across 63 categories
Agent modes	10
Perception	Screen vision
Memory	Persistent ("eternal") memory

The application runs a local HTTP server on `127.0.0.1:4096`, protected by HTTP basic authentication. The harness that drives SWE-bench talks to this server over the loopback interface. Because the server is local, the solving loop has no network egress of its own; the only outbound traffic is the model provider's API calls.

3.3 The model layer and honest naming

ZYRAXON-AI does not train its own model. It is a *scaffold* that drives externally provided language models. The provider used for this run is **OpenCode Zen** (provider id `opencode`, authenticated with `OPENCODE_API_KEY`). The provider offers a rotating catalogue of free and paid models, and the agent queries the provider's live configuration to discover which models are currently active. At the time of this run the active models included:

- `deepseek-v4.1-flash` (148)
- `deepseek-v4-flash` (83)
- `nemotron-3-ultra-free` (25)
- `fledge-alpha-free` (15)
- `nemotron-3.5-lightning-free` (12)
- `longcat-2.5-preview-free` (4)
- `ling-3.1-flash-free` (3)
- `mimo-v2.6-flash-free` (2)
- `space-bunny-free` (2)
- `big-pickle` (1)

The number in parentheses is how many of the 300 instances had their final frozen patch produced by that model; the model provider is probed live and the harness rotates models on failure, so patches are multi-model.

All ten identifiers are standard OpenCode Zen catalogue entries, listed in OpenCode's public model documentation: the eight with a `-free` suffix (and `big-pickle`) are offered at no cost, while `deepseek-v4.1-flash` and `deepseek-v4-flash` are paid-tier models on the same provider and endpoint (`https://opencode.ai/zen/v1`). No model is trained or hosted by ZYRAXON-AI; the agent's access to a given model is bounded only by the account's entitlements.

Some models are rate-limited or occasionally stop responding. The agent therefore treats the model as a pool, not a constant: it probes candidates, caches a healthy winner, and rotates on failure. The models that actually produced the final frozen patch during this run, with the number of instances each was responsible for, are listed above. Because more than one model was used, the leaderboard metadata reports `model_display: Multiple` and lists the models under `tags.model`.

We note explicitly, because earlier drafts of this work contained an error, that no model with the id `zyraxon-multi-free` exists. That identifier was an internal placeholder and has been removed; the real provider and model identifiers are the ones listed above.

3.4 The solving loop

For each instance the harness performs the following steps.

1. **Checkout.** A working tree is created at the instance's base commit. Later commits are pruned so that the fix cannot be recovered from history.
2. **Prompt.** The agent receives the issue text (the `problem_statement` field) and the path to the checkout. It does not receive `FAIL_TO_PASS`, `PASS_TO_PASS`, or `hints`.
3. **Explore and edit.** Using its tools, the agent reads files, searches the codebase, forms a hypothesis, and edits source files in place. It may run the project's tests locally as part of its own reasoning.
4. **Emit a patch.** The harness captures the working-tree diff as the prediction. New files are included by staging them before diffing.
5. **Retry on failure.** If the turn fails for infrastructure reasons—a model timeout or a dropped connection—the harness salvages any edits already made; if none exist, it rotates to another model and retries once.

A subtle but important detail is that the agent's own edits are *salvaged* when a turn dies. If the model has already modified files before going silent, those edits are kept rather than discarded. This avoids the pathological failure mode in which a nearly complete solution is thrown away because the connection dropped at the last moment.

3.5 Integrity constraints

The solver is subject to hard constraints that make the result meaningful.

- **No web browsing.** The solving loop has no browsing capability. It cannot reach GitHub, mirrors of the fixes, or any SWE-bench source.
- **No test knowledge.** The F2P and P2P lists are read only by the grader, after the patch is frozen. The solver never sees them.
- **No hints.** The `hints` field is not passed to the agent.
- **Fresh checkout per attempt.** A retry starts from the base commit again, so a failed first attempt cannot leak its partial state into the second.

4 The Evaluation Harness

Running 300 Docker evaluations is an exercise in systems engineering as much as machine learning. This section describes the harness and the safeguards that were necessary to make it reliable. These details matter for reproducibility, and they are the part of such projects that is least often written down.

4.1 Docker evaluation

Grading uses the official SWE-bench harness. For each instance the harness:

1. starts the prebuilt image `sweb.eval.x86_64.<instance_id>:latest`;
2. copies the candidate patch into the container and applies it inside `/testbed`;
3. runs the repository's official test command, capturing the output between the `>>> Start Test Output` and `>>> End Test Output` markers;
4. parses the result and computes `resolved` from the F2P and P2P sets.

A crucial correctness point is that the repository must *not* be bind-mounted over `/testbed`. Many repositories (for example `astropy`) bake a version string or an editable install into the image; mounting a host checkout over the image's tree corrupts that and causes every test to fail. The correct flow is to start the image, copy the patch in, and apply it inside the container. A second correctness point is that stdout and stderr must be merged when the log is captured, because the test markers are emitted on stderr while results are printed on stdout; capturing them separately reorders the markers and makes even the gold patch look unresolved.

4.2 The verdict ledger

Every verdict is written to an append-only ledger, `results/ledger.jsonl`. Each row binds an instance to a patch hash, a phase, a status, a patch size in bytes, a duration, and a timestamp. Because the ledger is bound to the patch hash, a stale verdict for an older patch can never be counted against a newer one. The scorecard recomputes the headline number from the ledger and the current patch set, and it explicitly counts and discards stale verdicts. In this run the scorecard discarded 27 legacy verdicts and confirmed that zero "passes" lacked a corresponding patch.

4.3 Resource guards

A long run must survive transient infrastructure failures. The harness therefore includes:

- a disk guard that prunes dangling Docker images when the volume fills (it never uses `docker image prune -a`, which would delete the tagged base images);
- a memory guard that checks `MemAvailable` and limits container memory;
- an inter-process lock so that concurrent verifications do not race;
- a watchdog that health-probes the local server and restarts it when it hangs.

These guards were not optional. During development the disk filled with orphaned overlay2 layers from killed containers, and the local server hung silently more than once. Restarting `dockerd` was what actually reclaimed the orphaned layers. Pulling eval images on demand from Docker Hub meant that a daemon restart lost no work.

4.4 Patch protection

Retries are the most dangerous part of the harness. An earlier implementation deleted a prediction before re-solving; if the re-solve produced an empty patch, or if the process was killed mid-retry, the previously good patch was lost permanently. The fix was to *copy* the good patch to a stash before re-solving and restore it only if the new attempt is empty, with a recovery pass at startup so that a crash mid-retry self-heals. This change alone recovered dozens of stranded patches.

5 Experimental Setup

This section records the concrete configuration in which the results were produced, so that a reader can judge what would and would not reproduce.

Table 2: Software and configuration used for the reported run.

Item	Value
System under test	ZYRAXON-AI
Agent runtime	Bun 1.3.14 / Electron 42
Local server	`http://127.0.0.1:4096` (HTTP basic auth)
Model provider	OpenCode Zen (`opencode`)
Attempts per instance	2 (best-of-2; see §10)
Evaluation harness	`swebench` 3.0.12 (classic venv)
Grader entry point	`swebench.harness.grading.get_eval_report`
Container runtime	Docker with prebuilt `sweb.eval.x86_64.*` images
Dataset	`princeton-nlp/SWE-bench_Lite` (300 test instances)

Two software-version facts are worth stating plainly. First, the evaluation harness used here is `swebench` 3.0.12, the version whose grader is invoked as `get_eval_report`; the current CLI (5.0.2) exposes a different command surface and does not yet ship the `submit` command group. Second, the Docker daemon required an MTU adjustment (1460) on this network to pull and start images reliably; without it, image pulls stalled intermittently, which manifested as spurious "no logs" instances rather than as evaluation failures.

5.1 Prompt and observation model

For each instance the agent receives exactly two pieces of information: the issue text (the `problem_statement` field) and the path to a checkout at the base commit. It receives no other benchmark field. In particular, the `FAIL_TO_PASS`, `PASS_TO_PASS`, and `hints` fields are withheld from the agent and read only by the grader after the prediction has been frozen.

The agent interacts with the checkout through its general tool surface: file reads, content searches, edits, and command execution. The prediction is the working-tree diff after the agent finishes, with newly created files staged so that they appear in the diff.

5.2 Retry and model rotation

A turn can fail for reasons that have nothing to do with the model's ability: a model may be rate-limited, a request may time out, or the local server may momentarily drop a connection. When that happens the harness does not count the instance as a failure. Instead it salvages any edits already present in the working tree; if none exist, it rotates to a different model and retries the instance once. This behaviour is the single most important thing to understand about the reported number, and it is why the run is labelled best-of-2.

6 Results

6.1 Overall

ZYRAXON-AI produced a non-empty patch for all 300 instances and all 300 were graded with real Docker.

Table 3: Overall result on SWE-bench Lite.

Metric	Value
Instances	300
Predictions produced	300
Graded (real Docker)	300
Resolved	**290**
Unresolved	10
Resolved rate	**96.67%**

The resolved rate is identical whether measured over all 300 instances or over the 300 that were graded, because every instance produced a patch.

6.2 By repository

Table 4: Resolved instances by source repository.

Repository	Resolved	Total
django/django	112	114
sympy/sympy	74	77
scikit-learn/scikit-learn	23	23
matplotlib/matplotlib	22	23
pytest-dev/pytest	16	17
sphinx-doc/sphinx	14	16
astropy/astropy	6	6
psf/requests	6	6
pylint-dev/pylint	6	6
pydata/xarray	5	5
mwaskom/seaborn	4	4
pallets/flask	2	3
Total	**290**	**300**

6.3 By issue year

Table 5: Resolved instances by the year the issue was filed.

Year	Resolved	Total
2012	1	1
2014	3	3
2015	1	1
2016	4	4
2017	15	16
2018	21	21
2019	58	59
2020	62	66
2021	40	42
2022	56	57
2023	29	30
Total	**290**	**300**

6.4 Failure analysis

The 10 unresolved instances are few and diffusely spread. The largest single contributor is `sympy` (3 failures), where issues involve deep symbolic algebra and a correct fix must match a subtle mathematical convention rather than merely making a test pass; `django` and `sphinx` follow with 2 failures each, and `matplotlib`, `pytest` and `flask` contribute one apiece. Even the weakest repository, `sphinx`, still resolves 14 of its 16 instances (87.5%).

Three failure modes dominate:

1. **Wrong-but-plausible fixes.** The agent changes behaviour in a way that satisfies the issue text but fails a P2P test, usually because it did not anticipate a call site elsewhere in the repository.
2. **Incomplete edits.** The fix touches one of several required locations.
3. **Environment-specific test failures.** A small number of instances fail for reasons unrelated to the patch's logic (for example, rendering or locale sensitivity in `matplotlib`).

We do not attempt to hide or reclassify these failures. They are part of the result.

7 Verification

The single most important property of a benchmark submission is that its result can be checked independently. ZYRAXON-AI's artifacts are designed so that every verdict can be re-derived *without* re-running Docker, and the whole result can be re-derived *with* Docker.

7.1 Re-deriving verdicts from recorded output

For every instance the repository stores the raw output of the official test command (`logs/<instance_id>/test`) together with the patch (`patch.diff`) and the verdict (`report.json`). The official grader function, `swebench.harness.grading.get_eval_report`, takes the patch and the recorded log and recomputes the verdict. We ran this across all 300 instances and obtained:

300/300 verdicts matched `results.json`; 0 mismatches; 290 resolved.

This is the strongest evidence we can offer that the published number is real: the number is not asserted, it is recomputed from the raw evidence by the benchmark's own code.

A detail that matters for anyone reproducing this: the grader requires both the `»»» Start Test Output` and `»»» End Test Output` marker lines to be present in the log, and it reads plain text rather than gzip. When extracting the recorded output, the marker lines must be kept.

7.2 Re-running the harness with Docker

The strongest end-to-end check is to re-run the official harness on the published predictions. Section A gives the exact commands. The resolved count must again be 290.

8 Threats to Validity

We enumerate the ways in which the reported result could be over-read, so that the reader does not have to.

8.1 Construct validity

The benchmark measures whether a patch makes the benchmark's chosen tests pass, not whether the patch is the *correct* fix. A patch can pass every F2P and P2P test while being semantically wrong in ways the tests do not cover, and conversely a correct fix can fail if it changes behaviour that an unrelated test happens to assert. Our number is therefore a statement about test-passing behaviour on this benchmark, exactly as the benchmark defines it, and nothing stronger.

8.2 Internal validity

The main internal threat is the retry mechanism. Because a second attempt is allowed, the reported number is an upper bound on what a single attempt would achieve. We have quantified this: 105 of 300 instances used the second attempt. A reader who wants a pass@1 figure should treat 96.67% as best-of-2 and not as pass@1.

A second internal threat is grader configuration. We mitigate it by re-deriving every verdict from the recorded raw output with the official grader and by keeping the two extraction details (marker lines retained; stdout and stderr merged) that are required for the grader to see the tests at all.

8.3 External validity

The run draws its models from a single provider, OpenCode Zen (a mix of free and paid tiers). Those models change over time, so the exact model mix is not guaranteed to reproduce. The artifacts are frozen and remain verifiable regardless. The result is also specific to the Python repositories in SWE-bench Lite and says nothing about other languages or domains.

8.4 Statistical validity

With 300 instances the standard error of the resolved rate is roughly $\sqrt{0.9667 \cdot 0.0333/300} \approx 1.0$ percentage points. Differences smaller than a few points between systems should not be treated as meaningful without repeated runs.

9 Reproducibility Checklist

✓ **Predictions public.** All 300 predictions are in `all_preds.jsonl` and under `logs/<id>/patch.diff`.

✓ **Verdicts public.** Each instance has a `report.json` produced by the official grader.

✓ **Raw test output public.** Each instance has `test_output.txt.gz`, the raw output of the official test command.

✓ **Reasoning traces public.** One trace per instance under `trajs/`.

✓ **Re-derivable.** Re-grading the raw output reproduces all 300 verdicts with zero mismatches.

✓ **No test knowledge.** F2P/P2P/hints are used only by the grader.

✓ **No web lookup.** The solver has no browsing capability.

× **pass@1.** This run is best-of-2, not pass@1; declared as such.

10 Limitations and Scope

10.1 This is best-of-2, not pass@1

We repeat this because it is the most important caveat in this report. The solving harness allows two attempts per instance. When the first attempt fails for infrastructure reasons, the harness rotates to a different model and retries. **105 of the 300 instances used the second attempt.** The reported 96.67% is therefore a best-of-2 number. It should not be compared naively against a pass@1 result. We set `system.attempts: "2+"` in the leaderboard metadata so that the SWE-bench team and readers are not misled.

We also note that the second attempt is not a hidden oracle: it uses a different model but the same prompt and the same lack of test knowledge. No benchmark test information is used to choose between attempts.

10.2 Provider dependence

The agent depends on the OpenCode Zen provider. Models are rate-limited and change over time, so an exact replication of this run may not reproduce the identical model mix. The artifacts (patches, logs, verdicts) are frozen and fully verifiable regardless; what cannot be guaranteed is that a fresh run today will use the same models.

10.3 Scope of the claim

The claim of this report is narrow and precise: *on the 300 instances of SWE-bench Lite, using the official Docker grader, ZYRAXON-AI's frozen set of predictions resolves 290 of them (96.67%).* We make no claim about other benchmarks, other splits, or other versions of the system.

11 Related Work

SWE-bench [1] established the issue-resolution task and the Docker evaluation methodology. SWE-bench Lite [2] provided the human-validated 300-instance subset used here. OpenHands [3] and SWE-agent [4] are agent scaffolds designed specifically for this kind of task; mini-SWE-agent is a minimal reference implementation. ZYRAXON-AI differs from these in that it is a general desktop agent whose tool surface and interface are built for interactive local use, and the benchmark run reported here is an application of that general agent rather than a purpose-built benchmark harness. On the model side, the models used here are drawn from the OpenCode Zen provider; the broader trend of open-weight coding models is what makes a fully local agent viable.

12 Conclusion

ZYRAXON-AI resolves 290 of 300 SWE-bench Lite instances (96.67%) using the official Docker grader, with no web access and no use of benchmark test knowledge. The result is fully reproducible: re-grading the published raw logs reproduces all 300 verdicts with zero mismatches. We have been explicit about the one way in which this result is weaker than a pass@1 number—it is best-of-2—and we have set the metadata to reflect that. All predictions, verdicts, logs, and reasoning traces are released with this report.

References

[1] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *International Conference on Learning Representations (ICLR)*, 2024.

[2] OpenAI. Introducing SWE-bench Lite. Technical report, 2024. https://openai.com/index/introducing-swe-bench-verified/

[3] X. Wang et al. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. 2024. https://arxiv.org/abs/2407.16741

[4] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press. SWE-agent: Agent–Computer Interfaces Enable Automated Software Engineering. 2024. https://arxiv.org/abs/2405.15793

A Exact Commands

Re-derive all verdicts without Docker

```
import gzip, json, os
from swebench.harness.utils import load_swebench_dataset
from swebench.harness.test_spec.test_spec import make_test_spec
from swebench.harness.grading import get_eval_report

res = set(json.load(open("results/results.json"))["resolved"])
data = {x["instance_id"]: x for x in
        load_swebench_dataset("princeton-nlp/SWE-bench_Lite", "test")}
matched = 0
for iid in os.listdir("logs"):
    d = os.path.join("logs", iid)
    if not os.path.isdir(d) or "__" not in iid:
        continue
    spec = make_test_spec(data[iid])
    pred = {"instance_id": iid, "model_name_or_path": "multiple",
            "model_patch": open(os.path.join(d, "patch.diff")).read()}
    log = gzip.open(os.path.join(d, "test_output.txt.gz"), "rt").read()
    open("/tmp/log.txt", "w").write(log)
    r = get_eval_report(spec, pred, "/tmp/log.txt", True)[iid]["resolved"]
    matched += (r == (iid in res))
print("matched:", matched)
```

Re-run the official harness with Docker

```
pip install swebench==3.0.12
python -m swebench.harness.run_evaluation \
  --dataset_name princeton-nlp/SWE-bench_Lite \
  --predictions_path all_preds.jsonl \
  --max_workers 4 --run_id zyraxon_check
```

B Complete Instance Table

Table 6 lists every one of the 300 instances with its repository and final verdict. The verdicts are the values recomputed from the published raw logs by the official grader.

Table 6: All 300 SWE-bench Lite instances and their verdicts.

Instance	Repo	Resolved
astropy__astropy-12907	astropy	yes
astropy__astropy-14182	astropy	yes
astropy__astropy-14365	astropy	yes
astropy__astropy-14995	astropy	yes
astropy__astropy-6938	astropy	yes
astropy__astropy-7746	astropy	yes
django__django-10914	django	yes
django__django-10924	django	yes
django__django-11001	django	yes
django__django-11019	django	no
django__django-11039	django	yes
django__django-11049	django	yes
django__django-11099	django	yes
django__django-11133	django	yes
django__django-11179	django	yes
django__django-11283	django	yes
django__django-11422	django	yes
django__django-11564	django	yes
django__django-11583	django	yes
django__django-11620	django	yes
django__django-11630	django	yes
django__django-11742	django	yes
django__django-11797	django	yes
django__django-11815	django	yes
django__django-11848	django	yes
django__django-11905	django	yes
django__django-11910	django	yes
django__django-11964	django	yes
django__django-11999	django	yes
django__django-12113	django	yes
django__django-12125	django	yes
django__django-12184	django	yes
django__django-12284	django	yes
django__django-12286	django	yes

Instance	Repo	Resolved
django__django-12308	django	yes
django__django-12453	django	yes
django__django-12470	django	yes
django__django-12497	django	yes
django__django-12589	django	no
django__django-12700	django	yes
django__django-12708	django	yes
django__django-12747	django	yes
django__django-12856	django	yes
django__django-12908	django	yes
django__django-12915	django	yes
django__django-12983	django	yes
django__django-13028	django	yes
django__django-13033	django	yes
django__django-13158	django	yes
django__django-13220	django	yes
django__django-13230	django	yes
django__django-13265	django	yes
django__django-13315	django	yes
django__django-13321	django	yes
django__django-13401	django	yes
django__django-13447	django	yes
django__django-13448	django	yes
django__django-13551	django	yes
django__django-13590	django	yes
django__django-13658	django	yes
django__django-13660	django	yes
django__django-13710	django	yes
django__django-13757	django	yes
django__django-13768	django	yes
django__django-13925	django	yes
django__django-13933	django	yes
django__django-13964	django	yes
django__django-14016	django	yes
django__django-14017	django	yes
django__django-14155	django	yes
django__django-14238	django	yes
django__django-14382	django	yes
django__django-14411	django	yes
django__django-14534	django	yes
django__django-14580	django	yes
django__django-14608	django	yes
django__django-14667	django	yes
django__django-14672	django	yes
django__django-14730	django	yes
django__django-14752	django	yes
django__django-14787	django	yes
django__django-14855	django	yes
django__django-14915	django	yes

Instance	Repo	Resolved
django__django-14997	django	yes
django__django-14999	django	yes
django__django-15061	django	yes
django__django-15202	django	yes
django__django-15213	django	yes
django__django-15252	django	yes
django__django-15320	django	yes
django__django-15347	django	yes
django__django-15388	django	yes
django__django-15400	django	yes
django__django-15498	django	yes
django__django-15695	django	yes
django__django-15738	django	yes
django__django-15781	django	yes
django__django-15789	django	yes
django__django-15790	django	yes
django__django-15814	django	yes
django__django-15819	django	yes
django__django-15851	django	yes
django__django-15902	django	yes
django__django-15996	django	yes
django__django-16041	django	yes
django__django-16046	django	yes
django__django-16139	django	yes
django__django-16229	django	yes
django__django-16255	django	yes
django__django-16379	django	yes
django__django-16400	django	yes
django__django-16408	django	yes
django__django-16527	django	yes
django__django-16595	django	yes
django__django-16816	django	yes
django__django-16820	django	yes
django__django-16873	django	yes
django__django-16910	django	yes
django__django-17051	django	yes
django__django-17087	django	yes
matplotlib__matplotlib-18869	matplotlib	yes
matplotlib__matplotlib-22711	matplotlib	no
matplotlib__matplotlib-22835	matplotlib	yes
matplotlib__matplotlib-23299	matplotlib	yes
matplotlib__matplotlib-23314	matplotlib	yes
matplotlib__matplotlib-23476	matplotlib	yes
matplotlib__matplotlib-23562	matplotlib	yes
matplotlib__matplotlib-23563	matplotlib	yes
matplotlib__matplotlib-23913	matplotlib	yes
matplotlib__matplotlib-23964	matplotlib	yes
matplotlib__matplotlib-23987	matplotlib	yes
matplotlib__matplotlib-24149	matplotlib	yes

Instance	Repo	Resolved
matplotlib__matplotlib-24265	matplotlib	yes
matplotlib__matplotlib-24334	matplotlib	yes
matplotlib__matplotlib-24970	matplotlib	yes
matplotlib__matplotlib-25079	matplotlib	yes
matplotlib__matplotlib-25311	matplotlib	yes
matplotlib__matplotlib-25332	matplotlib	yes
matplotlib__matplotlib-25433	matplotlib	yes
matplotlib__matplotlib-25442	matplotlib	yes
matplotlib__matplotlib-25498	matplotlib	yes
matplotlib__matplotlib-26011	matplotlib	yes
matplotlib__matplotlib-26020	matplotlib	yes
mwaskom__seaborn-2848	seaborn	yes
mwaskom__seaborn-3010	seaborn	yes
mwaskom__seaborn-3190	seaborn	yes
mwaskom__seaborn-3407	seaborn	yes
pallets__flask-4045	flask	yes
pallets__flask-4992	flask	no
pallets__flask-5063	flask	yes
psf__requests-1963	requests	yes
psf__requests-2148	requests	yes
psf__requests-2317	requests	yes
psf__requests-2674	requests	yes
psf__requests-3362	requests	yes
psf__requests-863	requests	yes
pydata__xarray-3364	xarray	yes
pydata__xarray-4094	xarray	yes
pydata__xarray-4248	xarray	yes
pydata__xarray-4493	xarray	yes
pydata__xarray-5131	xarray	yes
pylint-dev__pylint-5859	pylint	yes
pylint-dev__pylint-6506	pylint	yes
pylint-dev__pylint-7080	pylint	yes
pylint-dev__pylint-7114	pylint	yes
pylint-dev__pylint-7228	pylint	yes
pylint-dev__pylint-7993	pylint	yes
pytest-dev__pytest-11143	pytest	yes
pytest-dev__pytest-11148	pytest	yes
pytest-dev__pytest-5103	pytest	yes
pytest-dev__pytest-5221	pytest	yes
pytest-dev__pytest-5227	pytest	yes
pytest-dev__pytest-5413	pytest	yes
pytest-dev__pytest-5495	pytest	yes
pytest-dev__pytest-5692	pytest	yes
pytest-dev__pytest-6116	pytest	yes
pytest-dev__pytest-7168	pytest	yes
pytest-dev__pytest-7220	pytest	yes
pytest-dev__pytest-7373	pytest	yes
pytest-dev__pytest-7432	pytest	yes
pytest-dev__pytest-7490	pytest	no

Instance	Repo	Resolved
pytest-dev__pytest-8365	pytest	yes
pytest-dev__pytest-8906	pytest	yes
pytest-dev__pytest-9359	pytest	yes
scikit-learn__scikit-learn-10297	scikit-learn	yes
scikit-learn__scikit-learn-10508	scikit-learn	yes
scikit-learn__scikit-learn-10949	scikit-learn	yes
scikit-learn__scikit-learn-11040	scikit-learn	yes
scikit-learn__scikit-learn-11281	scikit-learn	yes
scikit-learn__scikit-learn-12471	scikit-learn	yes
scikit-learn__scikit-learn-13142	scikit-learn	yes
scikit-learn__scikit-learn-13241	scikit-learn	yes
scikit-learn__scikit-learn-13439	scikit-learn	yes
scikit-learn__scikit-learn-13496	scikit-learn	yes
scikit-learn__scikit-learn-13497	scikit-learn	yes
scikit-learn__scikit-learn-13584	scikit-learn	yes
scikit-learn__scikit-learn-13779	scikit-learn	yes
scikit-learn__scikit-learn-14087	scikit-learn	yes
scikit-learn__scikit-learn-14092	scikit-learn	yes
scikit-learn__scikit-learn-14894	scikit-learn	yes
scikit-learn__scikit-learn-14983	scikit-learn	yes
scikit-learn__scikit-learn-15512	scikit-learn	yes
scikit-learn__scikit-learn-15535	scikit-learn	yes
scikit-learn__scikit-learn-25500	scikit-learn	yes
scikit-learn__scikit-learn-25570	scikit-learn	yes
scikit-learn__scikit-learn-25638	scikit-learn	yes
scikit-learn__scikit-learn-25747	scikit-learn	yes
sphinx-doc__sphinx-10325	sphinx	yes
sphinx-doc__sphinx-10451	sphinx	yes
sphinx-doc__sphinx-11445	sphinx	yes
sphinx-doc__sphinx-7686	sphinx	yes
sphinx-doc__sphinx-7738	sphinx	no
sphinx-doc__sphinx-7975	sphinx	yes
sphinx-doc__sphinx-8273	sphinx	yes
sphinx-doc__sphinx-8282	sphinx	no
sphinx-doc__sphinx-8435	sphinx	yes
sphinx-doc__sphinx-8474	sphinx	yes
sphinx-doc__sphinx-8506	sphinx	yes
sphinx-doc__sphinx-8595	sphinx	yes
sphinx-doc__sphinx-8627	sphinx	yes
sphinx-doc__sphinx-8713	sphinx	yes
sphinx-doc__sphinx-8721	sphinx	yes
sphinx-doc__sphinx-8801	sphinx	yes
sympy__sympy-11400	sympy	yes
sympy__sympy-11870	sympy	yes
sympy__sympy-11897	sympy	yes
sympy__sympy-12171	sympy	no
sympy__sympy-12236	sympy	yes
sympy__sympy-12419	sympy	yes
sympy__sympy-12454	sympy	yes

Instance	Repo	Resolved
sympy__sympy-12481	sympy	yes
sympy__sympy-13031	sympy	yes
sympy__sympy-13043	sympy	yes
sympy__sympy-13146	sympy	yes
sympy__sympy-13177	sympy	yes
sympy__sympy-13437	sympy	yes
sympy__sympy-13471	sympy	yes
sympy__sympy-13480	sympy	yes
sympy__sympy-13647	sympy	yes
sympy__sympy-13773	sympy	yes
sympy__sympy-13895	sympy	yes
sympy__sympy-13915	sympy	yes
sympy__sympy-13971	sympy	yes
sympy__sympy-14024	sympy	yes
sympy__sympy-14308	sympy	yes
sympy__sympy-14317	sympy	yes
sympy__sympy-14396	sympy	yes
sympy__sympy-14774	sympy	yes
sympy__sympy-14817	sympy	yes
sympy__sympy-15011	sympy	yes
sympy__sympy-15308	sympy	yes
sympy__sympy-15345	sympy	yes
sympy__sympy-15346	sympy	yes
sympy__sympy-15609	sympy	yes
sympy__sympy-15678	sympy	yes
sympy__sympy-16106	sympy	yes
sympy__sympy-16281	sympy	yes
sympy__sympy-16503	sympy	yes
sympy__sympy-16792	sympy	yes
sympy__sympy-16988	sympy	yes
sympy__sympy-17022	sympy	yes
sympy__sympy-17139	sympy	yes
sympy__sympy-17630	sympy	yes
sympy__sympy-17655	sympy	yes
sympy__sympy-18057	sympy	yes
sympy__sympy-18087	sympy	yes
sympy__sympy-18189	sympy	yes
sympy__sympy-18199	sympy	yes
sympy__sympy-18532	sympy	yes
sympy__sympy-18621	sympy	yes
sympy__sympy-18698	sympy	yes
sympy__sympy-18835	sympy	yes
sympy__sympy-19007	sympy	yes
sympy__sympy-19254	sympy	yes
sympy__sympy-19487	sympy	yes
sympy__sympy-20049	sympy	yes
sympy__sympy-20154	sympy	yes
sympy__sympy-20212	sympy	yes
sympy__sympy-20322	sympy	yes

Instance	Repo	Resolved
sympy__sympy-20442	sympy	yes
sympy__sympy-20590	sympy	yes
sympy__sympy-20639	sympy	yes
sympy__sympy-21055	sympy	yes
sympy__sympy-21171	sympy	no
sympy__sympy-21379	sympy	yes
sympy__sympy-21612	sympy	yes
sympy__sympy-21614	sympy	yes
sympy__sympy-21627	sympy	yes
sympy__sympy-21847	sympy	yes
sympy__sympy-22005	sympy	no
sympy__sympy-22714	sympy	yes
sympy__sympy-22840	sympy	yes
sympy__sympy-23117	sympy	yes
sympy__sympy-23191	sympy	yes
sympy__sympy-23262	sympy	yes
sympy__sympy-24066	sympy	yes
sympy__sympy-24102	sympy	yes
sympy__sympy-24152	sympy	yes
sympy__sympy-24213	sympy	yes
sympy__sympy-24909	sympy	yes

C Multilingual Summary

This report is accompanied by a multilingual README set in the artifact repository, so that readers of many languages can understand the system and verify the result. Each README states the headline number, the integrity constraints, and the three verification paths. The languages covered are: Bengali, Chinese, Hindi, Spanish, Arabic, French, Russian, Portuguese, German, Japanese, Korean, Indonesian, Urdu, Turkish, Persian, Vietnamese, Thai, Italian, Dutch, Polish, and English.